

Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «Уральский федеральный университет имени первого
Президента России Б.Н. Ельцина»
Институт радиоэлектроники и информационных технологий-РТФ

ОТЧЕТ

По лабораторной работе №1

«Основы работы с Docker и PostgreSQL»

По дисциплине «Разработка приложений»

Выполнил: Лапушкин И. А.

Группа: РИМ-150950

Проверил преподаватель:
Кузьмин Д. И.

Екатеринбург

2025

Цель работы: Освоить фундаментальные концепции и базовые операции Docker: создание образов, запуск контейнеров, управление ими, работа с сетями и томами. На практике закрепить навыки, запустив изолированную базу данных PostgreSQL и подключившись к ней извне.

Задачи:

1. Установить и проверить работу Docker.
2. Изучить базовые команды Docker.
3. Запустить контейнер с PostgreSQL в изолированном режиме.
4. Запустить контейнер с pgAdmin и подключить его к контейнеру с БД через сеть Docker.
5. Подключиться к БД из pgAdmin, создать схему и выполнить запросы.
6. Обеспечить сохранность данных БД с помощью томов Docker.

Ход работы:

1. Установка и проверка Docker

С официального сайта был установлен Docker Desktop (Рисунок 1).

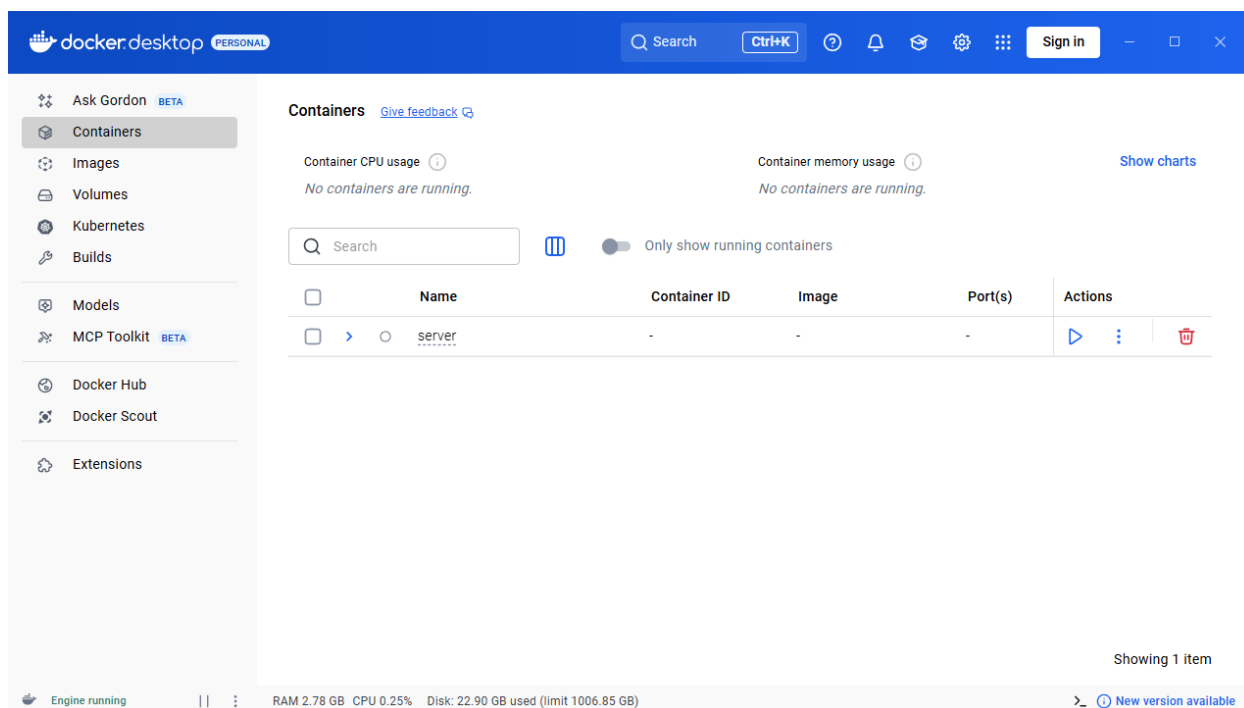


Рисунок 1 – Docker Desktop

Далее, запустив Pycharm и вписав в терминале команды «docker version», «docker compose version» и «docker hello-world», убедились в правильной работе Docker (Рисунки 2-3).

```
PS E:\Github rep\DevelopmentURFU2025> docker version
Client:
Version:           28.4.0
API version:       1.51
Go version:        go1.24.7
Git commit:        d8eb465
Built:             Wed Sep  3 20:59:40 2025
OS/Arch:           windows/amd64
Context:           desktop-linux

Server: Docker Desktop 4.46.0 (204649)
Engine:
Version:           28.4.0
API version:       1.51 (minimum version 1.24)
Go version:        go1.24.7
Git commit:        249d679
Built:             Wed Sep  3 20:57:37 2025
OS/Arch:           linux/amd64
Experimental:      false
containerd:
Version:           1.7.27
GitCommit:         05044ec0a9a75232cad458027ca83437aae3f4da
runc:
Version:           1.2.5
GitCommit:         v1.2.5-0-g59923ef
docker-init:
Version:           0.19.0
GitCommit:         de40ad0
PS E:\Github rep\DevelopmentURFU2025> docker compose version
Docker Compose version v2.39.2-desktop.1
PS E:\Github rep\DevelopmentURFU2025> 
```

Рисунок 2 – Команды «docker version» и «docker compose version»

```
PS E:\Github rep\DevelopmentURFU2025> docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
Digest: sha256:d4aaab6242e0cace87e2ec17a2ed3d779d18fbfd03042ea58f2995626396a274
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.
```

Рисунок 3 – Команда «docker run hello-world»

2) Базовые команды Docker. Работа с образами и контейнерами

Воспользуемся базовыми командами для просмотра информации (Рисунок 4).

```
PS E:\Github rep\DevelopmentURFU2025> docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
skeleton-bot-sber-bot latest             0aae26df5841       5 weeks ago        93.1MB
skeleton-bot-sber-backend latest            51bae7b87eb8       5 weeks ago        98.9MB
hello-world          latest            1b44b5a3e06a       4 months ago       10.1kB
yandex/clickhouse-server latest           c739327b5607       3 years ago        826MB

PS E:\Github rep\DevelopmentURFU2025> docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
PS E:\Github rep\DevelopmentURFU2025> docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
cca89b4e1d9c       hello-world         "/hello"            49 seconds ago     Exited (0) 48 seconds ago              vibrant_swanson
916d0844015e       yandex/clickhouse-server:latest "/entrypoint.sh"    16 months ago     Exited (0) 6 weeks ago              clickhouse-server

PS E:\Github rep\DevelopmentURFU2025>
```

Рисунок 4 – Ввод команд «docker images», «docker ps» и «docker ps -a»

Запустим один из существующих контейнеров и проверим его работоспособность в моём случае это контейнер с clickhouse (Рисунки 5-6).

```
PS E:\Github rep\DevelopmentURFU2025> docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
916d0844015e       yandex/clickhouse-server:latest "/entrypoint.sh"    16 months ago     Up 3 seconds        0.0.0.0:8123->8123/tcp, 0.0.0.0:9000->9000/tcp, 9009/tcp clickhouse-server

PS E:\Github rep\DevelopmentURFU2025>
```

Рисунок 5 – Запуск контейнера

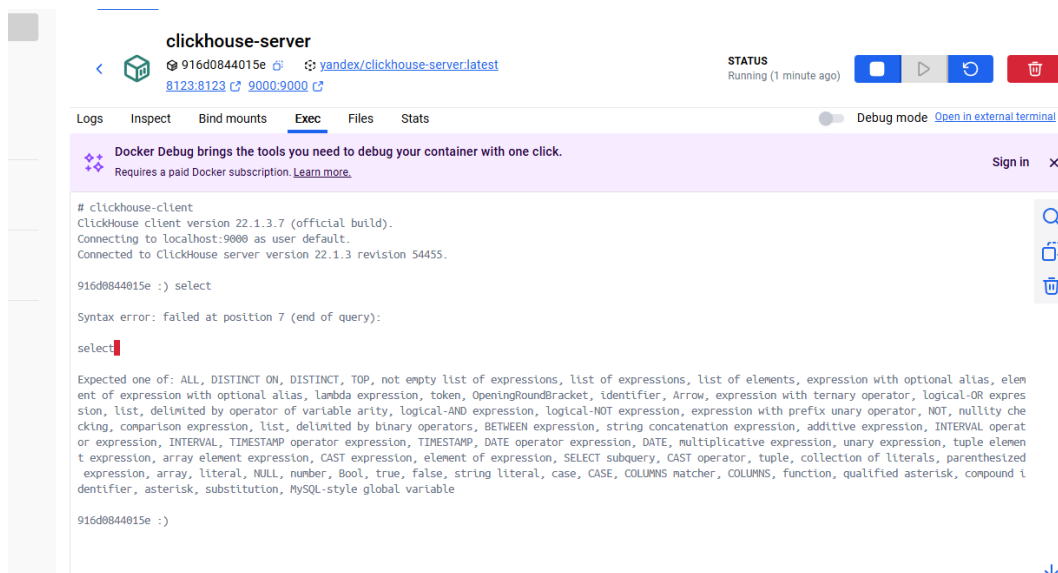


Рисунок 6 – Подключение к контейнеру через docker desktop

3) Запуск PostgreSQL в контейнере

Запустим контейнер с PostgreSQL и подключимся к БД из контейнера (Рисунок 7). Создадим таблицу и запишем тестовые данные (Рисунок 8).

```
087b178dc958: Pull complete
Digest: sha256:697ff70295146d53b702b07532fe68aaed7a5f10d04ff68edbeffd5fbacd15a5
Status: Downloaded newer image for postgres:15
3e45670408ac5f5343cffd36f344a15d562adcd135c2a69f5e778233a2de2f90
PS E:\Github rep\DevelopmentURFU2025> docker exec -it test_db psql -U user -d test_db
psql (15.15 (Debian 15.15-1.pgdg13+1))
Type "help" for help.

test_db=#
```

Рисунок 7 – Запуск контейнера и коннект к базе

```
test_db=# CREATE TABLE users (id SERIAL PRIMARY KEY, name VARCHAR(50));
CREATE TABLE
test_db=# INSERT INTO users (name) VALUES ('Alice'), ('Bob');
INSERT 0 2
test_db=# SELECT * FROM users;
 id | name
----+-----
  1 | Alice
  2 | Bob
(2 rows)

test_db=#
```

Рисунок 8 – Проверка созданной таблицы и вставка данных

4) Подключение к БД через pgAdmin из второго контейнера

Создадим сеть и проверим, что она появилась в списке сетей (Рисунок 9)

```
PS E:\Github rep\DevelopmentURFU2025> docker network create test_network
76a9bd8a80c973dc4198f755b370eb5dbf987d8ccc3305b763c79bfe0e066375
PS E:\Github rep\DevelopmentURFU2025> docker ls
docker: unknown command: docker ls

Run 'docker --help' for more information
PS E:\Github rep\DevelopmentURFU2025> docker network ls
NETWORK ID        NAME                DRIVER             SCOPE
5233f2f0108f      bridge              bridge             local
f8f7d2bc29e2      host                host               local
58f5f4f00e29      none                null               local
f25dba218ccc      server_default      bridge            local
76a9bd8a80c9      test_network        bridge            local
PS E:\Github rep\DevelopmentURFU2025>
```

Рисунок 9 – Создание сети

Подключим контейнер с PostgreSQL к сети и запустим pgAdmin в той же сети (Рисунок 10).

```
PS E:\Github rep\DevelopmentURFU2025> docker run --name pg_admin -- PGADMIN_DEFAULT_EMAIL=admin@example.com -- PGADMIN_DEFAULT_PASSWORD=1 -p 8080:80 --network test_network dpape/pgadmin4
Unable to find image 'dpape/pgadmin4:latest' locally
latest: Pulling from dpape/pgadmin4
014e56e61396: Pull complete
ba414809741b: Pull complete
65c1fa3fd4d6: Pull complete
72fb190ed6ea: Pull complete
94937133806c: Pull complete
7c802ae3d815: Pull complete
eb13c0937646: Pull complete
6a37ad29194b: Pull complete
2e0b10cc2685: Pull complete
a65690578e5b: Pull complete
15ab2d8956c2: Pull complete
492d1b24f14: Pull complete
9de99ec4bb9f: Pull complete
fe9d1dc9726: Pull complete
06949fa92b4c: Pull complete
c6538f601d4e: Pull complete
Digest: sha256:50700ac17936d0227f9e3e4bb080a91efb67064debc4b4737c35545bf1564088
Status: Downloaded newer image for dpape/pgadmin4:latest
a08f46e8fd0b84e9bc5e413143e8e3b799422fc80e11bedd1d63edc0031f9579
PS E:\Github rep\DevelopmentURFU2025>
```

Рисунок 10 – Подключение к сети и запуск pgAdmin

Откроем «localhost:8080» и попадем в pgAdmin и подключим сервер с нашей созданной базой данных, далее проверим, что все создано (Рисунок 11).

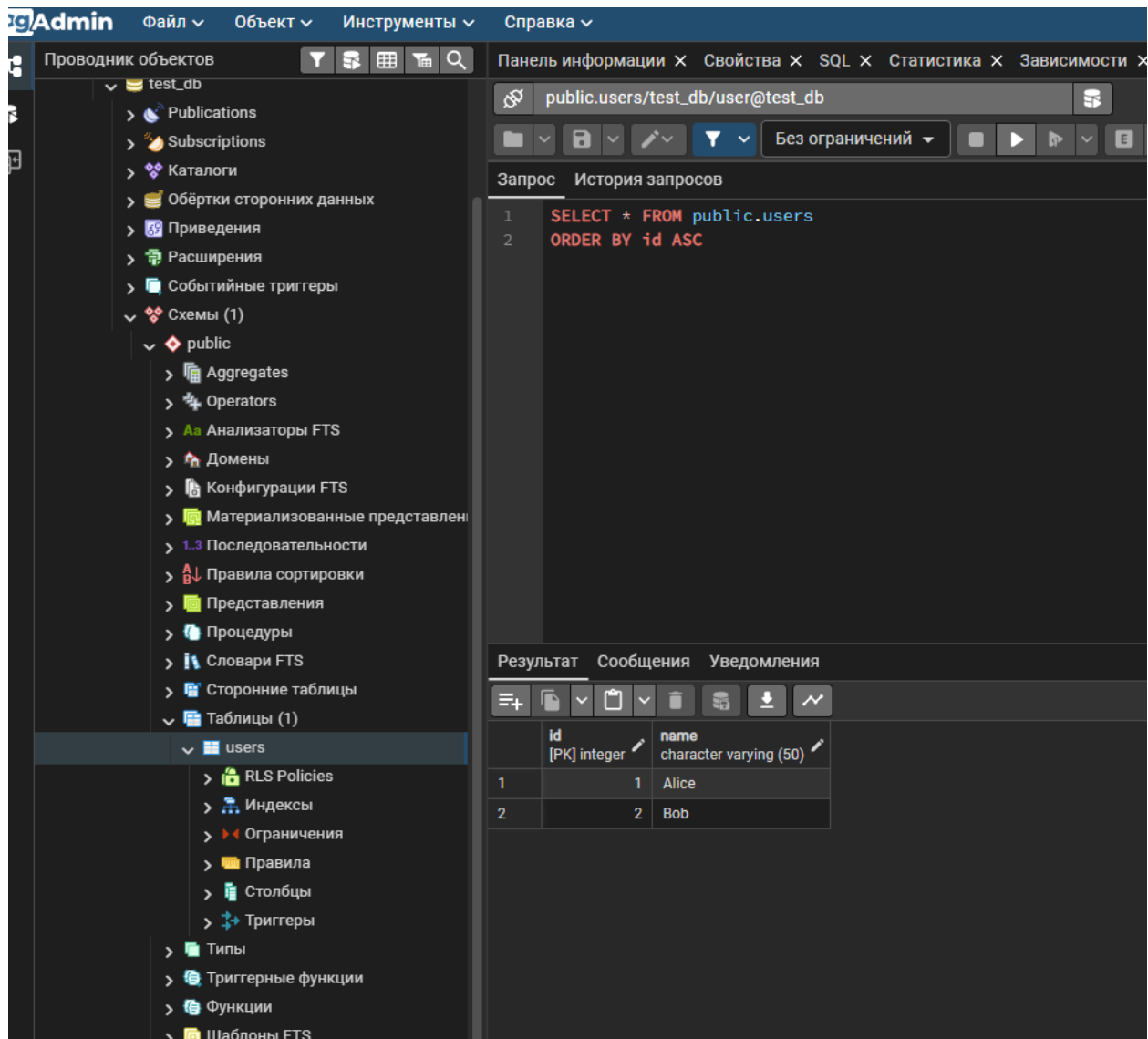


Рисунок 11 – Окно с pgAdmin

5) Сохранение данных с помощью Томов (Volumes)

Остановим текущий контейнер с БД, создадим том и запустим новый контейнер (Рисунок 12).

```
PS E:\Github rep\DevelopmentURFU2025> docker stop test_db
test_db
PS E:\Github rep\DevelopmentURFU2025> docker em test_db
docker: unknown command: docker em

Run 'docker --help' for more information
PS E:\Github rep\DevelopmentURFU2025> docker rm test_db
test_db
PS E:\Github rep\DevelopmentURFU2025> docker volume create postgres_db
postgres_db
PS E:\Github rep\DevelopmentURFU2025> docker volume ls
DRIVER      VOLUME NAME
local       34b1741b87b8744b1fc13e464a69ce1d9a939a480b778de4d87c49afd8453ef9
local       91c9de34f7f9da085d8749c6d418150e19bac8a2dd86388ff1a35101ee409b4
local       c4d8df3b009ff1c78edb185835f69fb1b07bba3b498125b09e2bfe9bca9d136
local       postgres_db
PS E:\Github rep\DevelopmentURFU2025> docker run -d --name test_db --e POSTGRES_USER=user --e POSTGRES_PASSWORD=1 --e POSTGRES_DB=test_db -p 5433:5432 -v postgres_db:/var/lib/postgresql/data --network test_network postgres:15
46ff46b3ce3d7d62364dd0c4318e73106cc7acf5d801615fe22a47fb0800cc24
PS E:\Github rep\DevelopmentURFU2025>
```

Рисунок 12 – Создание тома

Через pgAdmin подключимся к нашей БД, создадим таблицу и введем данные (Рисунок 13).

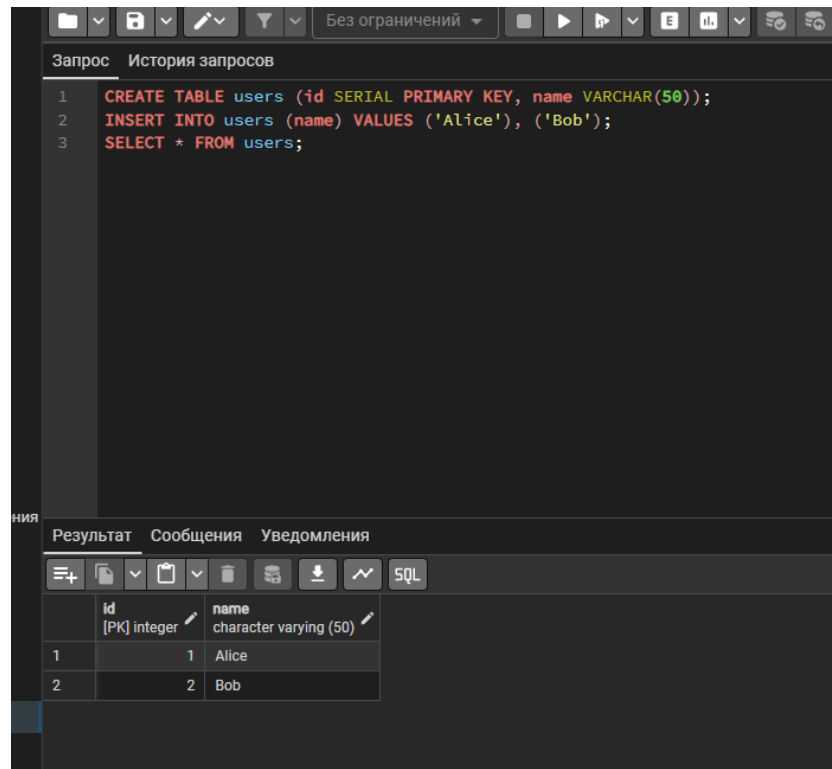


Рисунок 13 – Создание таблицы через pgAdmin

Далее отключим контейнеры, включим заново и проверим сохранность данных (Рисунки 14-15).

```
PS E:\Github rep\DevelopmentURFU2025> docker stop test_db
test_db
PS E:\Github rep\DevelopmentURFU2025> docker start test_db
test_db
```

Рисунок 14 – Запуск контейнеров

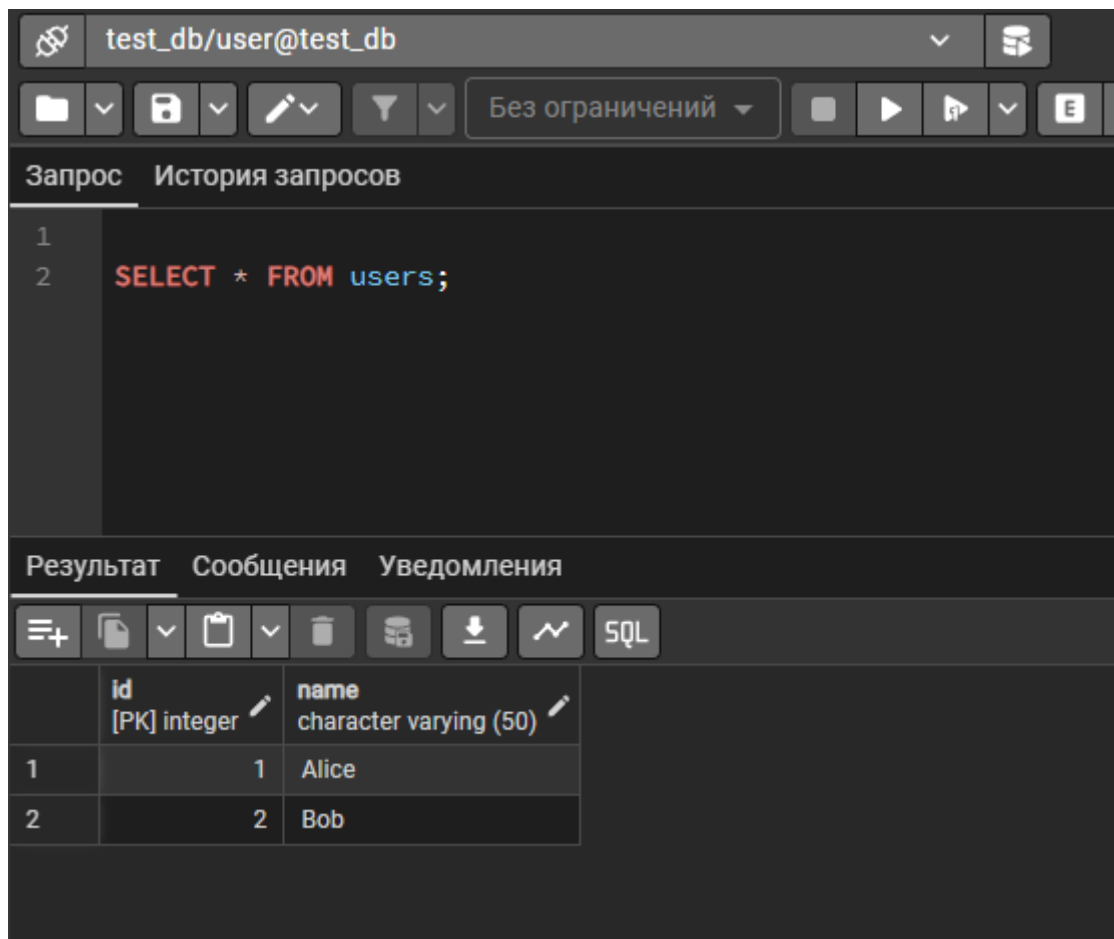
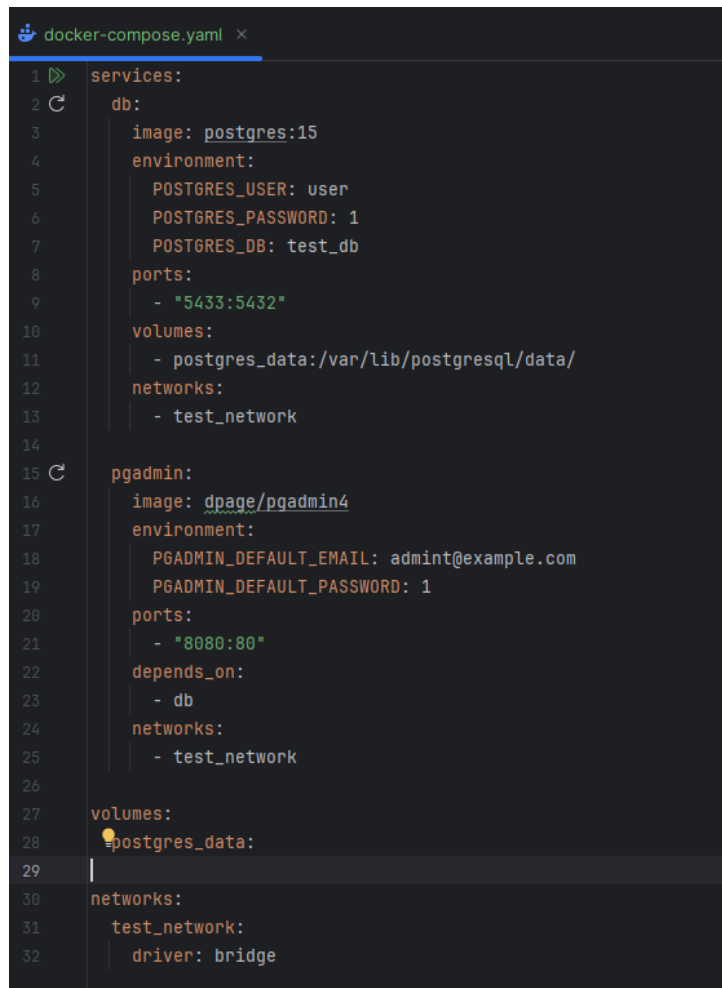


Рисунок 15 – Проверка данных в pgAdmin

Данные сохранились, всё работает.

6) Перенос конфигурации контейнеров в docker-compose.yml

Создадим файл docker-compose.yml (Рисунок 16). И подключимся к базе через пгадмин (Рисунок 17)



```
1 services:
2   db:
3     image: postgres:15
4     environment:
5       POSTGRES_USER: user
6       POSTGRES_PASSWORD: 1
7       POSTGRES_DB: test_db
8     ports:
9       - "5433:5432"
10    volumes:
11      - postgres_data:/var/lib/postgresql/data/
12    networks:
13      - test_network
14
15  pgadmin:
16    image: dpage/pgadmin4
17    environment:
18      PGADMIN_DEFAULT_EMAIL: admin@example.com
19      PGADMIN_DEFAULT_PASSWORD: 1
20    ports:
21      - "8080:80"
22    depends_on:
23      - db
24    networks:
25      - test_network
26
27  volumes:
28    postgres_data:
29
30  networks:
31    test_network:
32      driver: bridge
```

Рисунок 16 – Подключение БД к pgAdmin

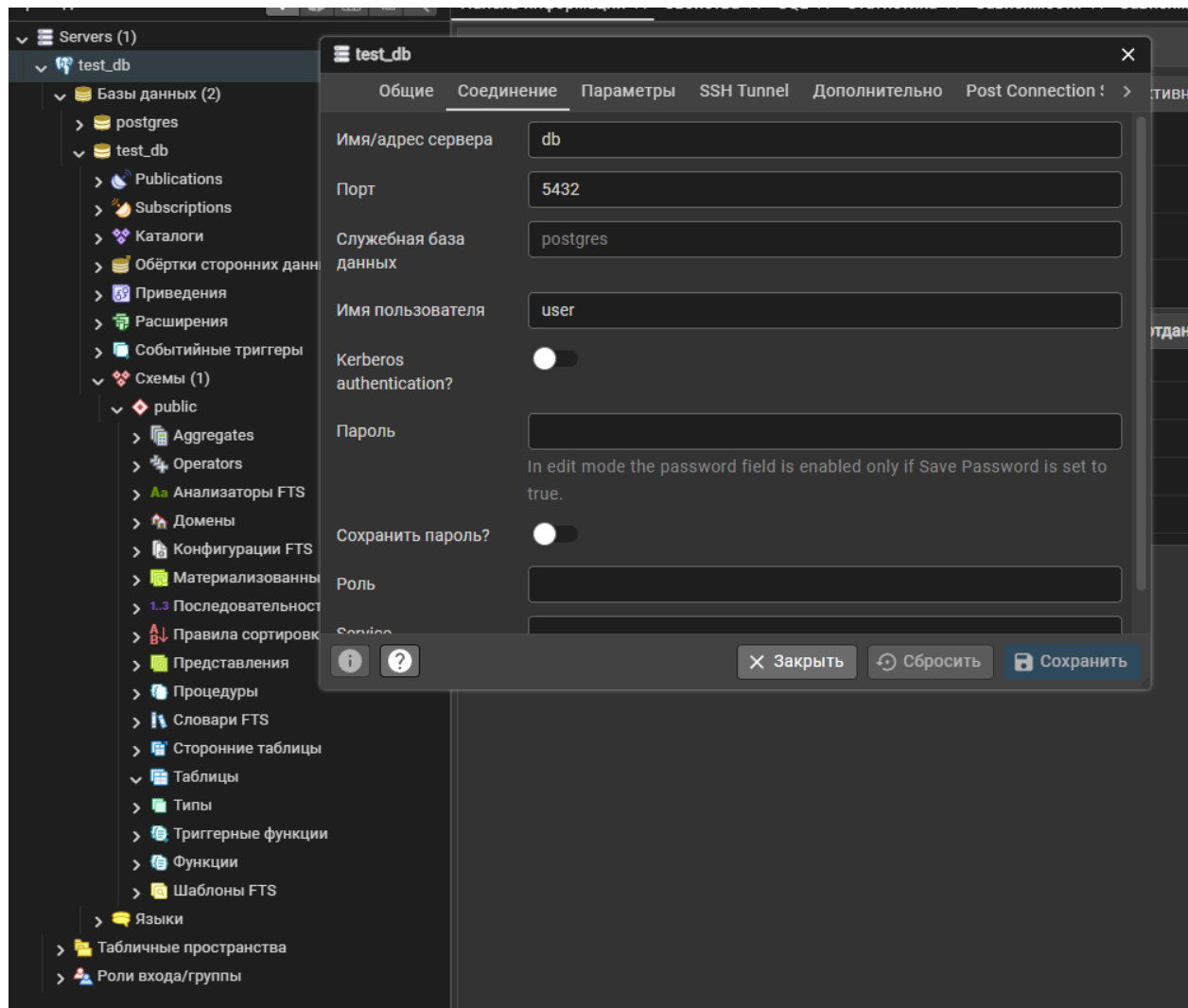


Рисунок 17 – Подключение БД к pgAdmin

Вопросы:

1. Что такое docker?

Docker – инструмент, позволяющий упаковывать приложения и всё, что им нужно для работы (код, библиотеки, настройки), в контейнеры. Например, применимо к нашей работе, мы можем запустить веб-сервер NGINX, СУБД PostgreSQL.

2. Для чего нужны тома и сети docker?

Том нужен для сохранения данных. Если подключить том, данные сохраняются даже после перезапуска или удаления контейнера. Сети нужны для обмена данными между контейнерами. По умолчанию контейнеры не видят друг друга, но если подключить их к одной созданной сети, они смогут общаться по имени

3. Как подключиться к контейнеру и выполнить в нём команды?

Мы можем подключиться к контейнеру через терминал, используя команду «`docker exec -it имя_контейнера команда`». В нашей работе мы использовали `docker exec -it test_db psql -U user -d test_db` для подключения к базе данных.

4. Для чего нужен pgAdmin?

PgAdmin – веб-интерфейс для управления PostgreSQL, он позволяет подключаться к базе данных через браузер, просматривать таблицы, схемы, индексы, выполнять SQL-запросы и создавать/удалять пользователей, базы, таблицы – без командной строки.

Вывод:

В ходе выполнения лабораторной работы были освоены основные концепции и инструменты Docker, необходимые для развертывания и управления приложениями в контейнерах.

Удалось успешно запустить контейнеры с веб-сервером NGINX и СУБД PostgreSQL, настроить тома (volumes) для сохранения данных базы данных даже после удаления контейнера, создать пользовательскую Docker-сеть, обеспечивающую взаимодействие между контейнерами по имени, развернуть веб-интерфейс pgAdmin и подключить его к PostgreSQL-контейнеру для удобного управления базой данных, автоматизировать запуск всей инфраструктуры с помощью файла docker-compose.yml.

В процессе работы были преодолены ошибки, связанные с отсутствием опыта работы с Docker, например, конфликты портов, некорректный синтаксис yaml, проблемы с аутентификацией и кавычками в SQL. Это позволило глубже понять особенности работы с Docker и важность внимательности при настройке конфигураций.

В результате создана полностью изолированная, воспроизводимая и переносимая среда разработки, состоящая из базы данных и инструмента для её администрирования, что подтверждает практическую применимость Docker в реальных проектах.

Ссылка на репозиторий:

<https://github.com/orfimorf/DevelopmentURFU2025>